GenAI Used: Gemini (access on KU Linux machine using Firefox)

Prompt Given:

A program is described by the following prompt:

"Using recursive, object based implementation in C++, generate a program to find the solution to sudoku puzzles. The program will take a partially solved sudoku puzzle in a txt file where blank spaces are indicated by underscores. There are five files provided, named "puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", and "puzzle5.txt". The sudoku puzzles are 9 by 9 grids divided into 3 by 3 subgrids. The program should use lists to store and iterate through a sudoku puzzle. There should be detailed and understandable comments explaining each code block, function, variable initialization, class initialization, or complex section.

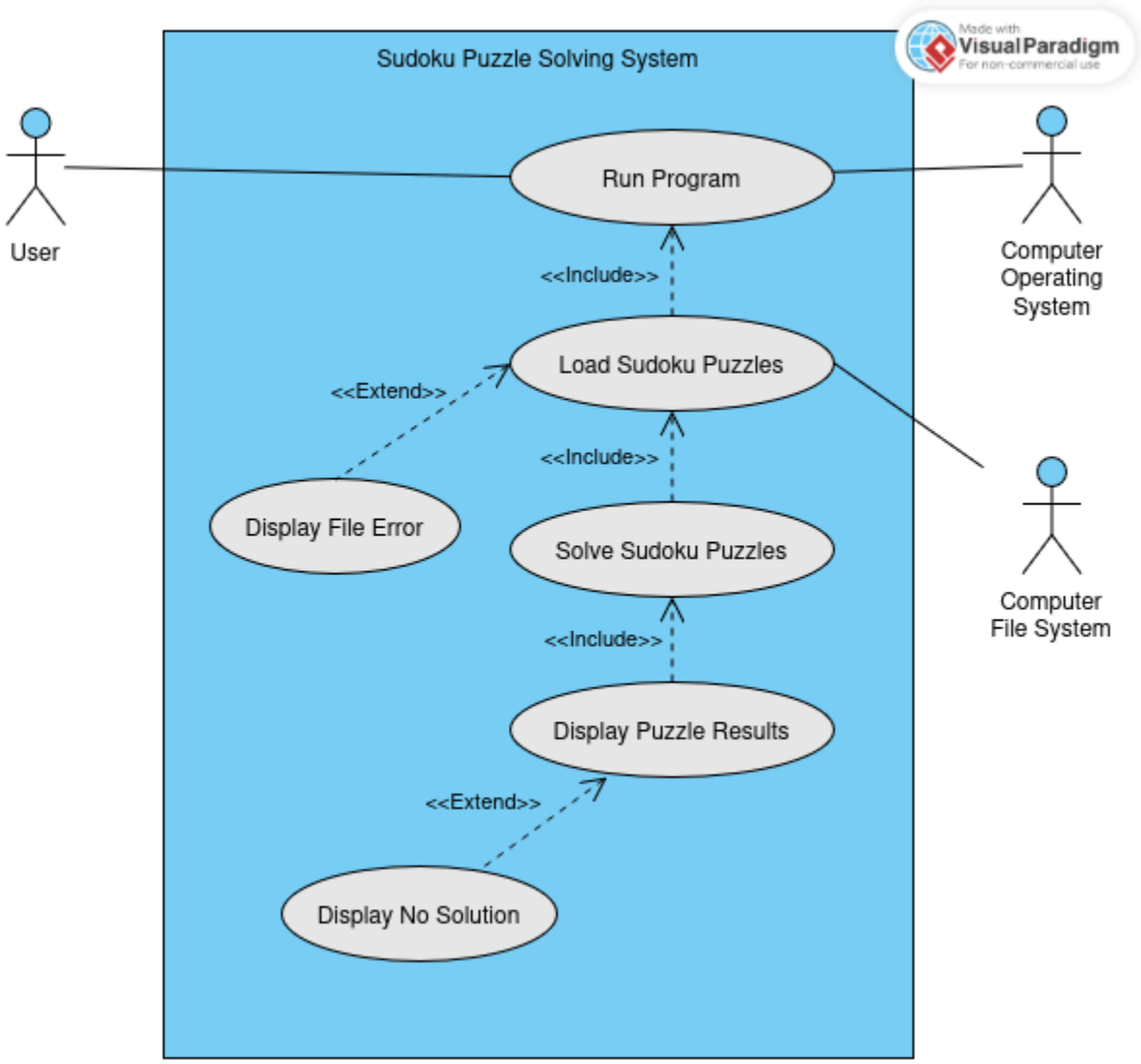
A puzzle should be solved with recursion through a depth-first search with backtracking; To solve a puzzle where k out of 81 cells are filled, the program should: Identify an empty cell and decide a list of values that could go in that empty cell. If none of the values could go in that empty cell (the values already appear in the same row, column, or subgrid), the result is a no-solution. Otherwise, if the first candidate value can go in the empty cell, place the first candidate value into the cell and move onto the next empty cell with a recursive function call. If the next empty cell returns a no-solution, we back track to the previous empty cell and use the next candidate value. This continues until the whole grid is filled successfully (k=81), or it has been determined that no solution exists (all candidate values return a no-solution).

The output for each puzzle should print: the filename of the puzzle, the unsolved puzzle, the puzzle solution (or solutions if multiple solutions exist), and the number of solutions. If no solution is found, print "no solution found" instead of the puzzle solution."

Given the description for the program above, create a UML Use Case Diagram using these steps for each use case involved in the overall program:

a. Identify/discover a use case
b. Provide a brief description for the use case
c. Provide a step-by-step flow (outline) of the events for the use case
d. Provide a detailed description for the use case

Graphic made with Visual Paradigm using GenAI output as guide:



➜ Use Case 1: Run Program

- Brief Description:
  - The use case allows the user to run the program that solves the sudoku puzzles.
- Outline:
  1. Navigate to correct directory
  2. Type command to run program
  3. Run command
- Detailed Description:
  - Navigate: Use the corresponding commands to navigate to the correct directory or use the file manager to open the terminal in the correct directory.
  - Type command: Type the corresponding command used to run files.

        - Run command: Running the command runs the executable file associated with the program, therefore running the program.

➜ Use Case 2: Load Sudoku Puzzle
- Brief Description:
    - The use case describes the process that the program uses to load the sudoku puzzles.
- Outline:
    1. Read puzzle file
    2. Store puzzle
    3. Create object
- Detailed Description:
    - Read puzzle file: The program reads one of the five files provided to it (puzzle1.txt - puzzle5.txt).
    - Store puzzle: The program stores the file contents into a 2D list.
    - Create Object: The program creates an object using the contents from the list created using puzzle file contents.

➜ Use Case 3: Solve Sudoku Puzzle
- Brief Description:
    - The use case describes the process that the program uses to solve the sudoku puzzles (a recursion based, object oriented, depth first search algorithm).
- Outline:
    1. Find empty cell
    2. Compare candidate value to cell surroundings
    3. If candidate valid, move to next cell
    4. If candidate invalid, backtrack to previous cell
- Detailed Description:
    - Find empty cell: The program iterates through the puzzle, checking if each cell is empty (does not have a number).
    - Compare candidate value to cell surroundings: If the program finds an empty cell, it will make sure that the candidate value that it plans to place there does not already exist in the same row, column or subgrid.
    - If candidate valid, move to next cell: If the candidate value is valid, the program will place it in that cell and repeat the process after finding the next empty cell.
    - If candidate invalid, backtrack to previous cell: If the candidate is invalid, the program will return to the previous cell, use the next candidate value, and return to the current empty cell.

➜ Use Case 4: Display Sudoku Puzzle
- Brief Description:
    - The use case describes the output that the program gives after solving the puzzles.
- Outline:
    1. Print puzzle filename

2. Print original puzzle
3. Print number of solutions
4. Print solutions if there are any

- Detailed Description:
  - Print puzzle filename: The program prints the filename of the solved puzzle.
  - Print original puzzle: The program prints the full unsolved puzzle.
  - Print number of solutions: The program prints the number of solutions found. There can be 0,1, or more solutions.
  - Print solutions if there are any: If solutions exist, the program will print each solved version of the sudoku puzzle. If no solutions exist, the program prints that no solutions exist.